A Combined Streaming FPGA Architecture for Client-Side CKKS Encoding and Encryption

Joseph Kurina
University of Washington
Bothell, Washington, USA
kurinj@uw.edu

Bogdan Pasca
Altera Corporation
Paris, France
bogdan.pasca@altera.com

Suleyman Demirsoy
Altera Corporation
London, U.K.
suleyman.demirsoy@altera.com

Sunwoong Kim
Rochester Institute of Technology
Rochester, New York, USA
sskeme@rit.edu

***Abstract*—Homomorphic encryption allows a computing service to process encrypted data, but the client must first encode and encrypt its inputs. Sharing hardware between floating-point encoding and modular encryption reduces area but requires those transform roles to execute at different times. We present a streaming FPGA architecture for CKKS encoding and encryption that computes the shared floating-point encoding transform once and performs the integer transforms for six residue moduli in parallel. Dedicated transform pipelines exchange intermediate results on chip rather than through off-chip memory. DSP Builder generates the transform RTL, and Altera HLS IP Gen compiler compiles the surrounding SYCL pipeline into IP integrated with DMA and control logic in an accelerator functional unit. At polynomial degree $N = 8192$, the Altera Agilex 7 implementation delivers 5,143 complete ciphertexts per second at 471 MHz. One SEAL-Embedded CPU worker on the same workstation delivers 113 ciphertexts per second for real-valued inputs using all 4,096 slots. Measurements cover prepared-input computation through complete ciphertext construction, including FPGA submission and output delivery.**

***Index Terms*—CKKS, homomorphic encryption, FPGA, SYCL, DSP Builder, streaming architecture, IFFT, NTT.**

I. Introduction

Homomorphic encryption (HE) allows a computing service to operate on encrypted data without decrypting it [1]. Applications include privacy-preserving neural-network inference [2], image processing [3], and speech recognition [4]. The data owner retains the secret key, prepares encrypted inputs, and decrypts the returned results. The Cheon, Kim, Kim, and Song (CKKS) scheme supports approximate arithmetic on packed real or complex values [5]. Packing places many values in one ciphertext so that the service can process them together.

Preparing that ciphertext requires substantial computation before the service can begin. The client maps a message vector to polynomial coefficients using a complex-valued transform, scales and rounds them to integers, and encrypts the polynomial. A residue number system (RNS) represents the large ciphertext modulus through several smaller primes, requiring modular arithmetic for every prime [6]. Client-side encoding and encryption therefore combine floating-point and integer workloads, whereas most existing HE hardware research addresses server-side ciphertext operations [7]–[14].

Existing client accelerators balance arithmetic reuse against concurrent execution. Aloha-HE [15] reuses butterfly units to reduce area, while ABC-FHE [16] uses parallel streaming cores with shared Fourier and modular transform arithmetic. The accelerator proposed in this paper uses a workstation-attached FPGA with dedicated parallel number theoretic transforms (NTTs) and shares only the modulus-independent inverse fast Fourier transform (IFFT). This partition removes competition for transform engines between moduli and between the secret-key and plaintext-plus-error paths. The resulting architecture trades arithmetic reuse for concurrent encoding and encryption. Encoded coefficients pass into encryption through on-chip buffers, avoiding an intermediate off-chip memory transfer. Here, *client-side* identifies the data-owner operation performed by the workstation, which supplies parameters, keys, and randomness, and assembles the ciphertext.

The integration challenge is to preserve coefficient order across transforms and prevent input or output storage from being overwritten while still in use. SYCL expresses device computation in C++, with FPGA extensions for typed on-chip FIFOs called pipes and imported register-transfer-level (RTL) components [17]. We use these mechanisms to connect RTL developed with Altera's DSP Builder [18] to SYCL polynomial-arithmetic tasks. Altera HLS IP Gen compiler [19] compiles the tasks and their pipes into one hardware pipeline. Import buffers store transform outputs, pipes carry ordered coefficients, and the host collects complete ciphertexts after accelerator completion. Each transform transfers four coefficients together through four parallel data paths, called *coefficient lanes*. These lanes carry parts of one polynomial. They do not encrypt four messages independently.

The main contributions are:

1. A streaming symmetric CKKS architecture at $N = 8192$ with one IFFT operating on complex doubles through four coefficient lanes, and six physical 30-bit RNS pipelines, each with two dedicated NTT engines.
2. A hybrid SYCL/RTL implementation generated with Altera HLS IP Gen compiler, with on-chip transform buffering and a host-memory DMA interface inside an accelerator functional unit (AFU).
3. An evaluation against one native SEAL-Embedded [20] worker running on the same host CPU.

The accelerator project repository is https://github.com/joekurina/FPT26_Agilex7_CKKS_Accelerator.

Accepted to special session track.

II. Background and Related Work

A. CKKS Encoding and Symmetric Encryption

CKKS represents plaintexts and ciphertext components in the negacyclic ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$, where N is a power of two and $Q = \prod_i q_i$ is a product of pairwise-coprime primes. Polynomial coefficients are integers modulo Q. Multiplication wraps powers of X^N to -1. RNS replaces arithmetic modulo the large Q with independent computations modulo each q_i [6]. We call each polynomial's representation modulo one prime an RNS *limb*.

Encoding takes $N/2$ complex plaintext values, called slots, and expands them into a conjugate-symmetric vector of length N. A negacyclic IFFT maps this vector to polynomial coefficients. Scaling by Δ and rounding produces an encoded integer plaintext polynomial m, which is reduced modulo each prime [5], [6]. The scale controls the representation of fractional values.

Symmetric encryption combines m with a small error polynomial e, the secret-key polynomial s, and a uniform polynomial a sampled in NTT representation. The NTT is the modular counterpart of the Fourier transform. It enables polynomial multiplication to be performed by efficient point-wise multiplication of coefficients. With i indexing the RNS modulus, define $\widehat{m+e}_i = \mathrm{NTT}_{q_i}(m+e)$ and $\widehat{s}_i = \mathrm{NTT}_{q_i}(s)$. The ciphertext components in NTT representation are

$$c_{1,i} = a_i, \tag{1}$$

$$c_{0,i} = \widehat{m+e}_i - \widehat{s}_i \odot a_i \pmod{q_i}, \tag{2}$$

where $\odot$ denotes point-wise coefficient multiplication. All per-modulus pairs $(c_{0,i}, c_{1,i})$ together represent one ciphertext $(c_0, c_1) \in R_Q^2$. The two NTTs in (2) can run independently, and each modulus admits a separate computation.

Both transforms use $O(N \log N)$ butterfly arithmetic, where each butterfly combines a small group of coefficients. The IFFT operates on complex doubles, whose real and imaginary components are each stored as a 64-bit double-precision floating-point value. The NTT operates in $\mathbb{Z}_{q_i}$ and uses a primitive $2N$-th root of unity for the negacyclic transform [21]. At $N = 8192$, each radix-2 transform has 13 stages. Our configuration uses six 30-bit primes satisfying $q_i \equiv 1 \pmod{2N}$. Modular multiplication outside the imported NTT uses Barrett reduction [22].

B. Related Work

Microsoft SEAL implements CKKS in software [23]. SEAL-Embedded, the baseline used here, reduces the memory requirements of client-side CKKS operations for embedded processors [20]. Hardware clients include CHOCO-TACO for encrypted compute offloading [24], RACE and RISE for RISC-V systems [25], [26], and the NTT accelerator integrated with SEAL-Embedded by Di Matteo *et al.* [27]. Software and protocol approaches also reduce client work by exploiting encoding redundancy [28] or using a bootstrapping-assisted client protocol [29].

TABLE I
Accelerator Parameters and Measured Clock

Parameter	Value
Polynomial degree N	8,192
Complex slots	up to 4,096
Transform stages	13
Coefficients per transfer	4
Transfers per polynomial	2,048
RNS moduli / physical modulus pipelines	6 / 6
Modulus width	30 bits
IFFT arithmetic	Complex double precision
Pipe residues	32 bits
Shared converted coefficients	Signed 64 bits
Concurrent complete operations	1
NTT engines	12
Measured application clock	471 MHz

Client accelerators make different choices about supported operations and arithmetic reuse. Lee *et al.* provide configurable CKKS encryption and decryption architectures [30]. Aloha-HE shares one double-precision FFT butterfly and three modular NTT butterflies across encoding, encryption, decryption, and decoding to reduce area [15]. ABC-FHE uses two reconfigurable streaming cores that share arithmetic between FFT and NTT modes [16]. Section IV-D compares these designs with our dedicated-transform partition and distinguishes their measurement boundaries.

HEXL-FPGA implements HE arithmetic primitives in SYCL, including NTT and key switching, rather than focusing on client-side encoding and encryption [31]. Our design combines DSP Builder-generated RTL transforms with SYCL kernels for these client operations.

III. Accelerator Architecture

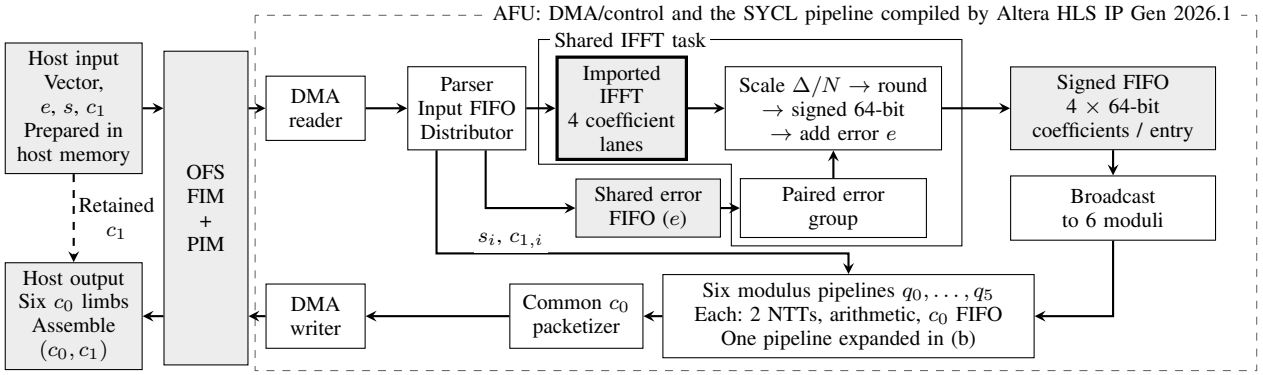
The accelerator shares one floating-point IFFT across six dedicated modular encryption pipelines. Table I summarizes the fixed configuration.

A. Overview and Design Parameters

Fig. 1 shows the complete dataflow and expands one of the modulus pipelines. One operation consumes a message vector and produces one complete ciphertext. The AFU admits one operation at a time while its internal arithmetic stages process coefficient streams concurrently. Within an operation, each transform accepts four coefficients per transfer: these are the four *coefficient lanes*. The six *modulus pipelines*, by contrast, process six representations of the same polynomial, one for each prime.

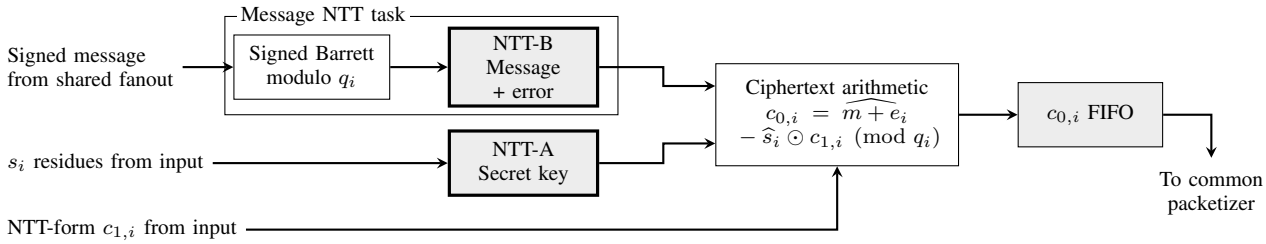
In Fig. 1(a), a DMA reader streams prepared inputs from host memory into the input parser. The encoding vector traverses the shared IFFT, followed by scaling, rounding, signed conversion, and error addition. A FIFO stores the converted coefficients for distribution to the six modulus pipelines. Secret-key coefficients and NTT-form c_1 bypass the IFFT. Each modulus pipeline reduces the shared coefficients, computes the two NTTs, and forms its c_0 limb as expanded in Fig. 1(b). Dedicated engines let the modulus computations

(a) Complete host–FPGA dataflow



Host control through OPAE SDK. One complete operation at a time; four coefficients per internal transfer.

(b) One physical modulus pipeline q_i (six spatial copies)



The six modulus pipelines produce the limbs of one ciphertext. Application clock: 470.8 MHz.
Thick borders: DSP Builder transform RTL. Other AFU blocks: SYCL tasks, storage, and DMA logic.

Fig. 1. CKKS client accelerator dataflow. (a) OFS FIM/PIM connects host memory to the AFU's DMA engines. The SYCL pipeline generated with Altera HLS IP Gen compiler shares the IFFT, conversion, and error addition across six modulus paths. Their c_0 limbs return through one packetizer for assembly with retained c_1. (b) Each modulus has independent secret-key and message NTTs; signed Barrett reduction precedes the message transform. Four coefficient lanes carry parts of one polynomial.

proceed without sharing a transform scheduler. A common output packetizer collects the six c_0 limbs for DMA delivery to host memory. The host combines them with retained c_1.

B. SYCL and the Hardware/Software Partition

DSP Builder generates the computationally intensive IFFT and NTT datapaths as RTL cores [18]. We package these cores as imported libraries and call them from SYCL tasks. Other tasks parse and distribute input, convert and broadcast coefficients, apply modular reduction, compute ciphertext arithmetic, and collect output. Altera HLS IP Gen compiler generates the hardware pipeline from this SYCL implementation, with FIFO pipes carrying intermediate coefficient groups on chip [19].

The generated pipeline IP is integrated into an AFU within Open FPGA Stack (OFS) [32]. The FPGA Interface Manager (FIM) supplies the platform interfaces, clocks, and resets [33]; the Platform Interface Manager (PIM) adapts its host channel to the AFU's memory and control interfaces [34]. The AFU contains the DMA engines, controller, and generated SYCL pipeline. A C++ host program prepares inputs with SEAL-Embedded and uses the OPAE SDK, the software development kit for the Open Programmable Acceleration Engine framework, to access the AFU and register host-memory buffers [35]. The AFU's RTL handles DMA and control; the generated pipeline combines DSP Builder transform RTL with SYCL-defined arithmetic and dataflow. The hardware pipeline remains resident on the FPGA. The AFU invokes it once per encryption operation, with its internal tasks executing concurrently.

C. Input Buffering and Shared IFFT

The input stage separates host-memory transfer from delivery to the arithmetic pipelines. The DMA reader supplies 512-bit words, which the parser assembles into groups containing four encoding values, their error samples, and the secret-key and c_1 coefficients for every modulus. A FIFO between the parser and distributor can hold a complete polynomial's input groups. The distributor sends each field to its consumer in coefficient order. Blocking pipe writes hold the current group until its destinations accept it, accommodating differences in consumer progress without overwriting unread input.

The imported IFFT RTL IP core is based on a Gentleman–Sande decimation-in-frequency butterfly [36]:

$$u = a + b, \tag{3}$$

$$v = (a - b) \cdot \omega, \tag{4}$$

where a, b, and ω are complex doubles. The four coefficient lanes carry one group of four complex values per accepted transfer, so a length-N vector requires 2,048 transfers.

The IFFT task pairs each input group with its error samples. As transform results return, it applies Δ/N, rounds the real components, converts them to signed 64-bit integers, and adds the errors. A FIFO sized for at least one complete polynomial holds these four-coefficient, 256-bit groups. The broadcast task reads each group once and delivers a copy to every modulus pipeline before advancing. Thus, the floating-point transform and integer conversion are shared across all six moduli, while the FIFO accommodates short-term differences in consumer progress.

D. DSP Builder Streaming Transform Architecture

Fig. 2 compares the IFFT and NTT pipelines and expands the transpose-memory mechanism that aligns operands between butterfly sections.

DSP Builder generates pipelines of butterfly sections, twiddle-factor arithmetic, and memory-based permutations. The 13 radix-2 stages are grouped into seven mixed-radix sections using $8192 = 2 \cdot 4^6$. Interstage routing supplies the operands needed by each section while keeping intermediate polynomials in on-chip storage.

1) Negacyclic IFFT: The IFFT begins with an input transpose and complex conjugation. Two parallel radix-2 butterflies feed six four-point butterfly sections, connected by general-twiddle blocks and transpose networks. A final transpose reorders the result. Within a four-point section, complex sums, differences, and fixed rotations implement the butterfly. The coefficient tables supply the general-twiddle multiplications between sections. All real and imaginary components use double-precision arithmetic. Conjugation at the output completes the unnormalized inverse transform, followed by a lane-specific, ROM-driven phase multiplication for the negacyclic transform. The SYCL stage then applies Δ/N.

2) Negacyclic NTT: The NTT begins with a transpose and a negacyclic twiddle multiplication/reduction layer. Six radix-4 butterfly sections alternate with general-twiddle and transpose blocks, followed by two parallel radix-2 butterflies. A modulus selector travels with the data and selects the twiddle factors and reduction constants for q_i. The butterfly adders and subtractors form integer sums and differences of their input coefficients. They retain extra bits for intermediate results, reduction around the twiddle multiplications limits their range, and final coarse/fine correction produces residues in $[0, q_i)$. The generated structure supports all six configured primes and is instantiated for each transform role in each modulus pipeline.

The transpose networks supply coefficients that arrive on different lanes or at different times to the same butterfly section. Lane-selection multiplexers reorder values within a transfer. Independently addressed dual-port memories hold earlier coefficients and read them with later arrivals in the order required by the next section. This storage implements permutations that cannot be completed by exchanging the four values of one transfer alone. Valid-data control and the NTT modulus selector remain aligned with the reordered coefficients.

E. Parallel Modulus Pipelines

Let j index a coefficient, x_j denote an IFFT output, and e_j denote the shared error sample. The shared IFFT task computes t_j once. Each modulus pipeline reduces it to the plaintext-plus-error coefficient $p_{i,j}$:

$$t_j = \mathrm{round}\left(\mathrm{Re}(x_j) \cdot \frac{\Delta}{N}\right) + e_j, \tag{5}$$

$$p_{i,j} = t_j \pmod{q_i}. \tag{6}$$

Signed Barrett reduction in the message-input adapter reduces t_j modulo q_i. NTT-B in Fig. 1(b) transforms the resulting plaintext-plus-error stream, while NTT-A independently transforms the secret-key stream. Their outputs and the supplied $c_{1,i}$ are buffered until corresponding coefficient groups can be combined.

Using the NTT notation of (2), the ciphertext arithmetic kernel computes

$$c_{0,i,j} = \widehat{m + e}_{i,j} - \widehat{s}_{i,j}\, c_{1,i,j} \pmod{q_i}. \tag{7}$$

Thus $c_{0,i}$ in Fig. 1 denotes the whole limb for modulus i, and $c_{0,i,j}$ in (7) denotes its coefficient j. All six modulus pipelines have separate NTT-A and NTT-B engines, giving twelve NTT engines in total. Complete-vector buffers align their outputs. Each pipeline writes its c_0 groups into a dedicated FIFO, and a common packetizer drains the limbs in modulus order.

F. RTL Integration and Flow Control

The imported transforms produce output that their downstream consumers may not immediately accept. A wrapper between each RTL core and its SYCL kernel packs coefficient groups, marks valid data, and buffers results until the next stage is ready. The C++ declaration and RTL import specification agree on the placement of the four coefficients and their control fields.

1) SYCL Pipeline Operation: The AFU starts one generated hardware pipeline for each admitted operation. Within that component, SYCL tasks implement input distribution, transforms, broadcast, modular reduction, ciphertext arithmetic, and output collection. The tasks communicate through blocking pipes and execute concurrently as data and output space become available. Their operation is coordinated in hardware; the host waits for the AFU's complete-operation result.

The IFFT and NTT tasks each wait for an input coefficient group before calling the imported RTL. Their wrappers capture valid transform outputs and deliver them as downstream consumers accept them. In modulus pipeline i, both NTT engines use the fixed selector for q_i, with a distinct prime assigned to each pipeline.

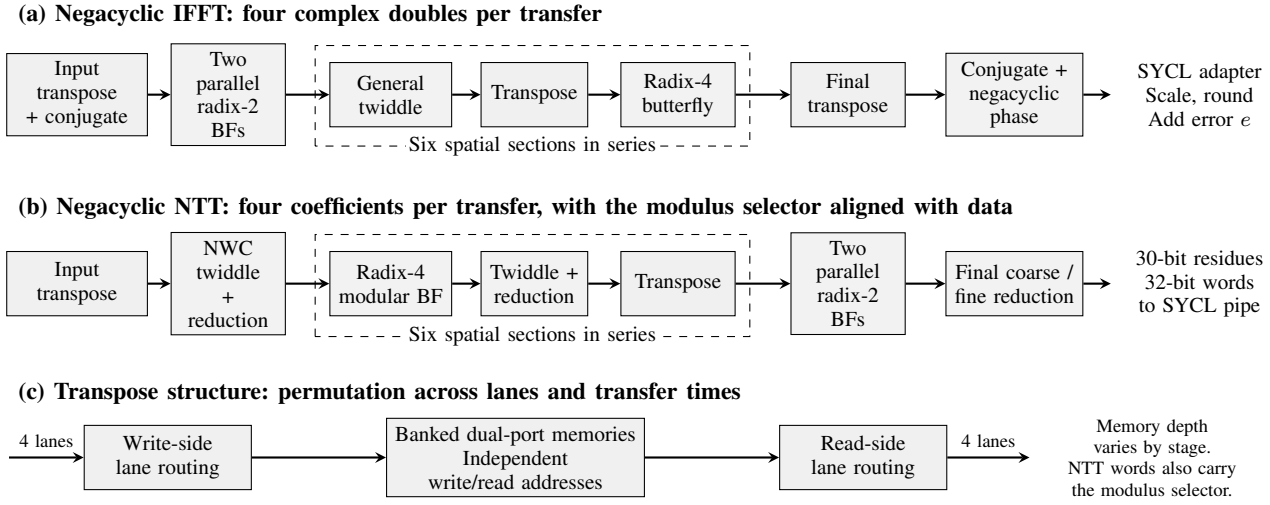


Fig. 2. DSP Builder IFFT and NTT pipelines and their transpose memories. BF denotes butterfly and NWC denotes negacyclic wraparound convolution. Each dashed group represents six spatial sections in series. The IFFT places radix-2 first and applies the negacyclic phase after the Fourier pipeline. Its SYCL output conversion scales by Δ/N, rounds, and adds e before broadcasting signed coefficients. The NTT places its NWC layer before the butterflies and radix-2 last. Panel (c) routes coefficients across lanes and transfer times, with memory widths and depths set by the stage. Import buffering is described in Section III-F.

2) Burst Capture and Output-Space Reservation: Each import wrapper has a 4,096-entry block-memory output FIFO, enough for two transforms of 2,048 coefficient groups each. This FIFO is separate from the complete-vector buffers between SYCL kernels. A nonempty FIFO presents its first entry until the downstream interface accepts it. Counters track accepted input groups and fully drained outputs to reserve space for results still in flight.

When the next stage stalls, the wrapper holds buffered output until it is accepted. It does not rely on directly stopping the generated arithmetic. The compiler treats each imported component as a variable-latency interface. Transfers follow valid/ready signals rather than a fixed cycle count. The downstream SYCL task forwards each completed return into its output pipe. This separates the transform's output burst from the rate at which later arithmetic can consume it, while preserving the four-coefficient grouping and order.

G. Host Runtime and Ciphertext Assembly

The host allocates and registers reusable input and output regions in host memory. For each operation, it copies the prepared input into the registered region, supplies the AFU with the DMA addresses and payload lengths, and starts execution through memory-mapped control registers. The DMA reader feeds the SYCL pipeline directly, and the DMA writer returns the packetized c_0 limbs to the registered output region. Intermediate transform data remains in on-chip buffers.

Completion requires both the arithmetic pipeline and output transfer to finish. The AFU applies a write-ordering fence before reporting the output ready, so the host can read the returned coefficients before reusing the registered regions. Collection expands the 32-bit residues into the ciphertext storage layout used by Microsoft SEAL and combines all six returned c_0 limbs with the retained c_1 limbs. The resulting ciphertext contains both polynomials in NTT representation with the configured scale and modulus chain. The measured host path repeats this sequence one complete ciphertext at a time.

IV. Experimental Results

Throughput is the number of complete ciphertexts returned per second over a long sequence, including the host work required to submit computation and construct outputs. One complete ciphertext contains both components across all six moduli.

A. Experimental Setup

The accelerator runs at 471 MHz on a BittWare IA-840F card with an Altera Agilex 7 AGFB027R25A2E2V FPGA. The software reference is one native SEAL-Embedded worker on the same Intel Xeon W-2295 workstation. The implementation uses the Altera HLS IP Gen compiler 2026.1, Quartus Prime Pro 26.1.1, and OFS FIM 2026.1-1. Host access uses OPAE SDK 2.14.0-3.

Both implementations use $N = 8192$, scale 2^{25}, the same key pair, and the six primes in Table II, taken from the SEAL-Embedded source code [37]. The accelerator uses 16 prepared real, partial, and complex cases; the CPU uses 16 real-valued variants with all 4,096 slots populated. Both receive prepared embeddings, sampled errors, uniform polynomials, and coefficient-domain keys. The input sets and delivery paths differ, so the CPU measurements provide a software reference rather than an identical-input speedup comparison.

The host timer covers IFFT, scaling and rounding, error addition, RNS reduction, both NTTs, ciphertext arithmetic, and complete ciphertext construction. FPGA timing also includes prepared-input copying, submission, completion, output delivery, and per-operation deadline checks. Input preparation,

TABLE II
FIXED PRIMES FOR THE SIX MODULUS PIPELINES AT $N = 8192$, FROM THE SEAL-EMBEDDED SOURCE CODE [37]

Pipeline i	Prime q_i
0	1,053,818,881
1	1,054,015,489
2	1,054,212,097
3	1,055,260,673
4	1,056,178,177
5	1,056,440,321

TABLE III
RESOURCE ATTRIBUTION FOR THE ACCELERATOR AND OFS PLATFORM

Metric	Total
ALMs needed	476,229
Logic ALUTs	538,822
Registers	1,071,132
Block-memory kbits	81,357
M20Ks	4,442
DSPs needed	3,056

allocation, warmup, and numerical verification are outside timing. The CPU copies c_1 into its output storage; the FPGA host assembles the returned c_0 with retained c_1.

Output storage is allocated before timing. The drivers retain the first, middle, and last outputs of each measured trial by exchanging buffer ownership. Ciphertext construction remains timed; decryption and decoding with unmodified Microsoft SEAL follow the timed interval.

The accelerator runs three trials at each of 1,024 and 8,192 operations. Each process qualifies the prepared corpus and completes 32 warmup operations before timing. The CPU runs three fresh processes at each of 1,024 and 8,192 operations, with 64 warmup operations per process. Reported rates divide total complete ciphertexts by total measured seconds.

B. Resource Utilization and Correctness

Table III reports the fitted resource attribution for the measured accelerator. The resources include the computational pipeline, DMA, control, and packet adaptation contained within the AFU region along with the platform infrastructure provided by the OFS FIM/PIM. Block-memory bits and M20K counts describe storage capacity and physical blocks, respectively.

The accelerator passes all numerical qualification tests. Every ciphertext passes all 4,096 decoded slots, including inactive slots, with canonical c_0/c_1 residues and exact retained-c_1 identity. Maximum absolute decoded error is 3.07×10^{-5} for the accelerator and 3.18×10^{-5} for the CPU, below the 0.1 threshold.

Numerical coverage excludes warmup outputs and unsampled measured outputs. The c_0 checks assess canonicality and use unmodified Microsoft SEAL for decryption and decoding.

C. Throughput and SEAL-Embedded Reference

Each record denotes one encoding-and-encryption operation. Table IV compares the runs of 1,024 and 8,192 records. FPGA / CPU is the ratio of the unrounded weighted rates, giving 45.25× and 45.35×, respectively, for the stated workloads.

D. Comparison with Prior Custom Accelerators

Table V compares the operation scope and reported results of client custom accelerators. The following discussion relates their arithmetic organization to our dedicated encoding/encryption path.

Aloha-HE shares one FFT butterfly and three NTT butterflies across encoding, encryption, decryption, and decoding to reduce area [15]. Our one double-precision IFFT and twelve NTT engines keep the encoding/encryption transforms physically separate. This removes transform scheduling conflicts between the two NTT roles and the six moduli, at the cost of more resident arithmetic. Lee *et al.*'s configurable encryption/decryption hardware accepts an already encoded polynomial for the reported encryption rate [30]. Our pipeline also performs the preceding encoding and passes its coefficients directly to encryption on chip.

ABC-FHE already exploits parallel streaming. Its two reconfigurable cores share arithmetic between FFT and NTT modes, use custom 55-bit floating point, and generate randomness and twiddles on chip [16]. Our fixed symmetric configuration retains double-precision encoding output and dedicates each NTT to one role and modulus, with host-prepared randomness and ROM twiddles. SYCL connects those independent RTL paths to the surrounding arithmetic and DMA streams. ABC-FHE supports larger bootstrappable parameters and evaluates them with an ASIC cycle model. Our results measure a physical FPGA at the stated fixed configuration.

In both our design and Aloha-HE, the modulus limbs collectively represent one ciphertext. Our host-observed throughput, Aloha-HE's whole-system latency, Lee *et al.*'s encryption-module rate, and ABC-FHE's cycle model use different operation scopes and parameters. Table V therefore provides architectural context rather than a normalized performance ranking.

E. Discussion and Limitations

The accelerator sustains approximately 5,143 complete ciphertexts/s while admitting one operation at a time. Within each operation, the shared IFFT feeds dedicated, concurrent NTT paths for all six moduli. The measurements include host submission and output construction, so they characterize the complete accelerator path rather than an isolated transform. Stage-level measurements are needed to determine which arithmetic or transfer stage limits this rate.

Larger polynomial degrees require regenerated transforms, coefficient tables, permutations, and storage. Longer modulus chains could reuse the IFFT result across successive modulus groups, with additional replay storage and aligned auxiliary

TABLE IV
THROUGHPUT COMPARISON OF CPU AND FPGA IMPLEMENTATIONS

Records per run	CPU weighted ct/s	CPU trial range	FPGA weighted ct/s	FPGA trial range	FPGA / CPU
1,024	113.42	113.38–113.46	5,133	5,125–5,139	45.25×
8,192	113.43	113.38–113.48	5,144	5,057–5,195	45.35×

TABLE V
CLIENT ACCELERATOR ARCHITECTURES AND MEASUREMENT BOUNDARIES

System	Operation and parameters	Platform	Reported result	Measurement scope
This work	Dedicated IFFT/NTTs; symmetric encoding/encryption; $N = 8192$; six concurrent 30-bit moduli; prepared real/complex inputs	Altera Agilex 7; 471 MHz	5,143 complete ciphertexts/s	Prepared-input copy through AFU execution, output delivery, and host ciphertext construction; input preparation excluded
Aloha-HE [15]	Public-key encoding/encryption; $N = 8192$, 3×54-bit	AMD Kintex-7 / MicroBlaze; 200 MHz	1.87 ms	Whole-system latency: software, hardware, and transfers
Lee *et al.* [30]	Public-key encryption; $N = 8192$, level 5; widths {30,20,20,20,20,20,30} bits	AMD XCU250; 250 MHz	2,950 encryption operations/s	Encryption-module rate, starting with an encoded polynomial
ABC-FHE [16]	Encoding/encryption; $N = 65536$, 36-bit primes, 24 levels	28-nm ASIC design; 600 MHz	1,112× over Core i7-12700 / Lattigo	Cycle model; absolute latency shown only in a plot

streams. Wider primes require revised arithmetic widths, reduction logic, and interfaces. These extensions remain implementation and validation work.

V. CONCLUSION

The accelerator connects one shared IFFT with four coefficient lanes and twelve dedicated NTT engines across six modulus pipelines through SYCL pipes. DSP Builder supplies the transform arithmetic, and Altera HLS IP Gen compiler generates the surrounding pipeline for an OFS-integrated AFU. At 471 MHz on Agilex 7, the accelerator delivers 5,143 complete ciphertexts/s with input preparation excluded. One SEAL-Embedded CPU worker on the same workstation delivers 113 ciphertexts/s for real-valued inputs using all 4,096 slots. All qualification ciphertexts and retained measured samples pass complete-ciphertext and all-slot checks. Future work will extend the architecture to larger polynomial degrees and wider RNS primes, requiring regenerated transforms, revised modular arithmetic, and evaluation at the new parameters.

ACKNOWLEDGMENTS

This material is based upon work supported by the National Science Foundation under Grant No. 2347253. The authors would like to thank Altera Corporation for providing the FPGA development board used in this work.

REFERENCES

[1] J. H. Cheon *et al.*, "Introduction to Homomorphic Encryption and Schemes," Chapter in Protecting Privacy through Homomorphic Encryption, pp. 3-28, Springer International Publishing, 2022.

[2] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy," in *Proc. ICML*, 2016.

[3] S. Kim and W. Cho, "Accelerating Homomorphic Comparison Operations for Thresholding Using an Asymmetric Input Range and Input Scaling," in *Proc. GLSVLSI*, 2024.

[4] D. L. Elworth, and S. Kim, "HEKWS: Privacy-preserving Convolutional Neural Network-based Keyword Spotting With a Ciphertext Packing Technique," in *Proc. MMSP*, 2022.

[5] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," in *Proc. ASIACRYPT*, 2017.

[6] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A Full RNS Variant of Approximate Homomorphic Encryption," in *Proc. SAC*, 2019.

[7] W. Zhao, Q. Chen, Y. Wang, H. Zhang, Z. Lu, and G. Qu, "A Survey on FPGA-based Accelerators for CKKS," in *Proc. ITC-Asia*, 2024.

[8] M. S. Riazi, K. Laine, B. Pelton, and W. Dai, "HEAX: An Architecture for Computing on Encrypted Data," in *Proc. ASPLOS*, 2020.

[9] R. Agrawal, L. De Castro, G. Yang, C. Juvekar, R. Yazicigil, and A. Chandrakasan, "FAB: An FPGA-Based Accelerator for Bootstrappable Fully Homomorphic Encryption," in *Proc. HPCA*, 2023.

[10] N. Samardzic *et al.*, "F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption," in *Proc. MICRO*, 2021.

[11] Y. Yang, H. Zhang, S. Fan, H. Lu, M. Zhang, and X. Li, "Poseidon: Practical Homomorphic Encryption Accelerator," in *Proc. HPCA*, 2023.

[12] Y. Zhu *et al.*, "DAHE: Parameter-Adaptive and Memory Efficient FPGA Acceleration of Homomorphic Encryption," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2687-2701, Aug. 2025.

[13] S. Kim, K. Lee, W. Cho, J. H. Cheon, and R. A. Rutenbar, "FPGA-based Accelerators of Fully Pipelined Modular Multipliers for Homomorphic Encryption," in *Proc. ReConFig*, 2019.

[14] S. Kim, K. Lee, W. Cho, Y. Nam, J. H. Cheon, and R. A. Rutenbar, "Hardware Architecture of a Number Theoretic Transform for a Bootstrappable RNS-based Homomorphic Encryption Scheme," in *Proc. FCCM*, 2020.

[15] F. Krieger, F. Hirner, A. C. Mert, and S. S. Roy, "Aloha-HE: A Low-Area Hardware Accelerator for Client-Side Operations in Homomorphic Encryption," in *Proc. DATE*, 2024.

[16] S. Yune *et al.*, "ABC-FHE: A Resource-Efficient Accelerator Enabling Bootstrappable Parameters for Client-Side Fully Homomorphic Encryption," in *Proc. DAC*, 2025.

[17] Intel Corporation, "Intel oneAPI DPC++/C++ Compiler Handbook for Intel FPGAs," Intel documentation. [Online]. Available: https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/developer-guide/.

[18] Altera, "DSP Builder," [Online]. Available: https://www.altera.com/products/development-tools/quartus-prime/dsp-builder.

[19] Altera, "HLS IP Gen Handbook," [Online]. Available: https://docs.altera.com/r/docs/m615048/current/hls-ip-gen-handbook/1-altera-hls-ip-gen-handbook.

[20] D. Natarajan and W. Dai, "SEAL-Embedded: A Homomorphic Encryption Library for the Internet of Things," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2021, no. 3, pp. 756–779, 2021.

[21] P. Longa and M. Naehrig, “Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography,” in *Proc. CANS*, 2016.
[22] P. Barrett, “Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor,” in *Proc. CRYPTO*, 1986.
[23] Microsoft, “Microsoft SEAL,” [Online]. Available: https://github.com/microsoft/SEAL.
[24] M. van der Hagen and B. Lucia, “Client-Optimized Algorithms and Acceleration for Encrypted Compute Offloading,” in *Proc. ASPLOS*, 2022.
[25] Z. Azad, G. Yang, R. Agrawal, D. Petrisko, M. Taylor, and A. Joshi, “RACE: RISC-V SoC for En/Decryption Acceleration on the Edge for Homomorphic Computation,” in *Proc. ISLPED*, 2022.
[26] Z. Azad, G. Yang, R. Agrawal, D. Petrisko, M. Taylor, and A. Joshi, “RISE: RISC-V SoC for En/Decryption Acceleration on the Edge for Homomorphic Encryption,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst*, vol. 31, no. 10, pp. 1523-1536, Oct. 2023.
[27] S. Di Matteo, M. Lo Gerfo, and S. Saponara, “VLSI Design and FPGA Implementation of an NTT Hardware Accelerator for Homomorphic SEAL-Embedded Library,” *IEEE Access*, vol. 11, pp. 72498–72508, 2023.
[28] A. S. Molahosseini and H. Vandierendonck, “Exploiting Data Redundancy in CKKS Encoding for High-Speed Homomorphic Encryption,” in *Proc. ASIA CCS*, 2024.
[29] A. Aikata, F. Krieger, and S. S. Roy, “Privacy at Your Fingertips: Enabling Rapid Client-Side Operations in Fully Homomorphic Encryption,” in *Proc. EuroS&P*, 2026.
[30] J. Lee, P. N. Duong, and H. Lee, “Configurable Encryption and Decryption Architectures for CKKS-Based Homomorphic Encryption,” *Sensors*, vol. 23, no. 17, Art. no. 7389, Aug. 2023.
[31] Y. Meng *et al.*, *Intel Homomorphic Encryption (HE) Acceleration Library for FPGAs (Version 2.0)*, 2022. [Online]. Available: https://github.com/intel/hexl-fpga.
[32] Altera, “Open FPGA Stack (OFS),” [Online]. Available: https://www.altera.com/products/software-tools/open-fpga-stack.
[33] Intel Corporation, “Open FPGA Stack Product Brief,” ver. 1.4, p. 2. [Online]. Available: https://docs.altera.com/v/u/docs/766116/open-fpga-stack-product-brief.
[34] Open FPGA Stack, “AFU Developers: Connecting an AFU to a Platform,” [Online]. Available: https://github.com/OFS/ofs-platform-afu-bbb/blob/master/plat_if_develop/ofs_plat_if/docs/PIM_AFU_interface.md.
[35] OPAE, “OPAE C API Programming Guide,” [Online]. Available: https://opae.github.io/latest/docs/fpga_api/prog_guide/readme.html.
[36] W. M. Gentleman and G. Sande, "Fast Fourier transforms: For fun and profit," in *Proc. November 7–10, Fall Joint Comput. Conf.*, 1966.
[37] D. Natarajan and W. Dai, *SEAL-Embedded*, 2021. [Online]. Available: https://github.com/microsoft/SEAL-Embedded.